ACTIVIDAD NO. 1: PATRONES DE CREACIÓN
REUTILIZACIÓN DE SOFTWARE

Sistema para procesar informes con el patrón Abstract Factory

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
DEIVIS EDUARD RAMÍREZ MARTÍNEZ

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
2 DE NOVIEMBRE
2026

TABLA DE CONTENIDO

1. INTRODUCCIÓN 3
2. EJERCICIO 1: VENTAJAS E INCONVENIENTES DEL PATRÓN 3
2.1 Implementación sin patrón y sus problemas 4
2.2 Solución con Abstract Factory 5
2.3 Ventajas 5
2.4 Inconvenientes 6
2.5 Alternativas consideradas 6
2.6 Reutilización de software en la solución 7
3. EJERCICIO 2: DISEÑO E IMPLEMENTACIÓN 7
3.1 Participantes del patrón 7
3.2 Diagrama UML del sistema base 7
3.3 Implementación en Java 8
3.4 Decisiones de diseño 11
3.5 Resultados de la ejecución 12
4. EJERCICIO 3: EVOLUCIÓN DEL SISTEMA 12
4.1 Evolución 1: nueva categoría MIXTA 12
4.1.1 Diagrama UML ampliado: categoría MIXTA 13
4.2 Evolución 2: nuevo tipo PEDIDO en la categoría ENTRADA 14
4.2.1 Diagrama UML ampliado: tipo PEDIDO 15
4.3 Análisis del impacto de las evoluciones 17
5. DESARROLLO ADICIONAL: MÁS ALLÁ DEL ENUNCIADO 18
5.1 Etapa 4: categorías descubiertas con ServiceLoader 18
5.2 Pruebas automatizadas, incluida una prueba de arquitectura 20
5.3 Experimento de escalabilidad: ¿cuánto cuesta evolucionar? 21
5.4 Alternativa implementada: registro por clave 23
5.5 Guía de decisión y recomendación 24
6. CONCLUSIONES 24
7. BIBLIOGRAFÍA 26
8. ANEXO: ENTREGA DEL CÓDIGO Y EJECUCIÓN 26
9. AGRADECIMIENTO 26

FICHA DEL DOCUMENTO

Proyecto	Sistema para procesar informes (patrón Abstract Factory)
Documento	Actividad No. 1: Patrones de creación
Asignatura	Reutilización de Software
Versión	1.0 (noviembre de 2026)
Fecha	Noviembre de 2026
Autor	Alejandro De Mendoza
Docente	Deivis Eduard Ramírez Martínez
Repositorio	https://github.com/AlejoTechEngineer/abstract-factory-procesador-informes
Lenguaje	Java (JDK 8 o superior; probado con JDK 17 y 21)

1. INTRODUCCIÓN

La presente actividad tiene como propósito aplicar los patrones de diseño de creación estudiados en los Temas 3 y 4, mediante el análisis, diseño e implementación de un sistema de procesamiento de informes. El caso de estudio contempla dos dimensiones de clasificación: la categoría del informe (ENTRADA y SALIDA) y su tipo (FACTURA y COMPRA). Cada combinación representa un comportamiento específico de procesamiento, lo que plantea un desafío de diseño relacionado con la organización del código, la separación de responsabilidades y la capacidad de incorporar nuevas funcionalidades sin generar modificaciones extensivas en la estructura existente.

Para abordar este escenario, se estudia el patrón de diseño Abstract Factory, evaluando su pertinencia como mecanismo para encapsular la creación de familias de objetos relacionados y mantener la independencia entre los clientes y las implementaciones concretas. Asimismo, se contrasta esta solución con una implementación que no utiliza patrones de diseño, con el propósito de identificar sus diferencias en términos de acoplamiento, mantenibilidad, extensibilidad y complejidad de evolución.

El desarrollo se organiza de acuerdo con los tres ejercicios establecidos en el enunciado. El Ejercicio 1 presenta el análisis comparativo de las ventajas, los inconvenientes y las alternativas de aplicación del patrón Abstract Factory frente a una solución convencional. El Ejercicio 2 desarrolla el diseño de la solución mediante diagramas UML y su correspondiente implementación en Java. Posteriormente, el Ejercicio 3 plantea la evolución del sistema mediante la incorporación de una nueva categoría de informes (MIXTA) y un nuevo tipo de informe (PEDIDO), analizando los cambios requeridos en la arquitectura y su impacto sobre los componentes existentes. Adicionalmente, el capítulo 5 amplía voluntariamente el alcance de la actividad mediante la incorporación de mecanismos de descubrimiento de implementaciones con ServiceLoader, una suite de pruebas automatizadas que incluye verificaciones funcionales y de arquitectura, y un experimento orientado a cuantificar el esfuerzo de evolución del sistema. También se realiza una comparación con una alternativa de implementación, utilizando métricas obtenidas directamente de los archivos y las diferentes etapas del código fuente. De esta manera, el análisis no se limita a una descripción conceptual del patrón, sino que incorpora evidencia técnica para evaluar su comportamiento y las implicaciones de su aplicación.

Finalmente, el código fuente completo, organizado por etapas y acompañado de las instrucciones de compilación y ejecución, se encuentra disponible en el siguiente repositorio:

Repositorio de GitHub: https://github.com/AlejoTechEngineer/abstract-factory-procesador-informes

La implementación se realiza en Java; sin embargo, los principios de diseño, las relaciones estructurales representadas mediante UML y los fundamentos del patrón Abstract Factory son aplicables a otros lenguajes de programación orientados a objetos.

2. EJERCICIO 1: VENTAJAS E INCONVENIENTES DEL PATRÓN

En este ejercicio se analiza la aplicación del patrón de diseño Abstract Factory en el sistema de procesamiento de informes, identificando sus principales ventajas e inconvenientes frente a una implementación convencional. Se examina su impacto en la organización del código, el acoplamiento, la mantenibilidad y la extensibilidad, con el

propósito de determinar en qué medida este patrón aporta beneficios al diseño y evolución de la solución propuesta.

2.1 Implementación sin patrón y sus problemas

La solución más directa es un único procesador que interpreta el nombre del archivo (CAT_TIPO_nombre.txt) y decide con condicionales qué hacer. Esta es la versión base (4 combinaciones), tal como se entrega en sinpatron/ProcesadorSinPatronBase.java:

```
public static void procesarInformes(String[] reports) {
    for (String nombre : reports) {
        String[] p = nombre.split("_");
        String cat = p[0];
        String tipo = p[1];
        if (cat.equals("ENT")) {
            if (tipo.equals("FAC")) {
                System.out.println("Procesando informe de ENTRADA de tipo FACTURA");
            } else if (tipo.equals("COM")) {
                System.out.println("Procesando informe de ENTRADA de tipo COMPRA");
            } else {
                System.out.println("Tipo desconocido: " + tipo);
            }
        } else if (cat.equals("SAL")) {
            // ... análoga a ENT; después un else final ...
        }
    }
}
```

Funciona, pero presenta problemas estructurales que se agravan con la evolución exigida por el enunciado:

- **Violación del principio Abierto/Cerrado:** Añadir la categoría MIXTA y el tipo PEDIDO obliga a **reabrir y modificar el mismo método ya probado** (Tabla 1). Cada evolución arriesga regresiones en combinaciones que ya funcionaban.
- **Crecimiento multiplicativo y concentración de lógica:** El método debe conocer todas las combinaciones categoría × tipo (4 al inicio, 7 tras la evolución) y contener el procesado de todas ellas; la complejidad crece con el producto de ambas dimensiones, no con la suma.
- **Acoplamiento con cadenas de texto:** La decisión depende de comparar *strings* repartidos en ramas anidadas; un error tipográfico o una rama olvidada solo se detecta en ejecución.
- **Baja cohesión y pruebas difíciles:** No hay objetos que representen un informe: no se puede probar el procesado de una sola combinación sin pasar por el selector, ni reutilizar su lógica.
- **Ninguna garantía de coherencia.** Nada obliga a que todas las categorías soporten el mismo conjunto de tipos; esa regla vive solo en la disciplina del programador.

Con el propósito de evidenciar el impacto de incorporar nuevas funcionalidades en una implementación convencional, la Tabla 1 presenta los cambios requeridos en la versión sin patrón, considerando las modificaciones realizadas sobre el código fuente de cada etapa. Esta comparación permite identificar el esfuerzo de adaptación de la solución y las implicaciones que tiene su evolución sobre la estructura existente.

Tabla 1. Efecto de la evolución sobre la versión sin patrón (medido sobre sinpatron/).

Métrica	Versión base	Tras MIXTA + PEDIDO	Variación
Líneas del procesador	35	46	+11
Comparaciones equals() en el selector	6	10	+4
Combinaciones categoría × tipo en un único método	4	7	+3
Métodos existentes que hubo que reescribir	—	1 (procesarInformes)	siempre el mismo

2.2 Solución con Abstract Factory

Según el Tema 3, *Abstract Factory* proporciona una interfaz para crear **familias de productos relacionados** sin especificar sus clases concretas, y se recomienda cuando se dispone de una familia de productos y se prevé incorporar nuevas familias en el futuro. Ambas condiciones se dan aquí. La correspondencia elegida es:

- **Familia = categoría** (ENTRADA, SALIDA y, después, MIXTA): una fábrica concreta por categoría.
- **Producto = tipo de informe** (FACTURA, COMPRA y, después, PEDIDO): una interfaz de producto abstracto por tipo.

¿Por qué esta orientación y no la inversa? El enunciado dice que «para cada categoría hay diferentes tipos» y que la nueva categoría MIXTA contendrá «los dos tipos», es decir, una familia completa. Con una fábrica por categoría, añadir MIXTA es el caso favorable del patrón (nueva fábrica, sin tocar las existentes). Con una fábrica por tipo, añadir MIXTA obligaría a modificar *todas* las fábricas.

Ficha del patrón (los cuatro elementos que definen un patrón de diseño: nombre, problema, solución y consecuencias):

- **Nombre:** Abstract Factory (Fábrica Abstracta), patrón de creación del catálogo GoF, conocido también como *kit* o *toolkit*.
- **Problema:** crear familias de objetos relacionados o dependientes sin acoplar el cliente a sus clases concretas. Aquí, informes de distintas categorías (ENTRADA, SALIDA, MIXTA) cuyos tipos (FACTURA, COMPRA, PEDIDO) deben ser coherentes y poder evolucionar sin reescribir el procesador.
- **Solución:** una interfaz de fábrica (`InformeFactory`) declara una operación de creación por tipo; cada categoría implementa su fábrica concreta y el cliente solo usa las interfaces.
- **Consecuencias:** aísla las clases concretas, facilita el intercambio de familias y promueve la consistencia entre productos; a cambio, añadir productos nuevos obliga a modificar la fábrica abstracta y las concretas.

Analogía con el ejemplo visto en clase: En el ejemplo de la asignatura la familia es el tema visual (claro u oscuro) y los productos son el botón y la casilla de verificación. Aquí la familia es la categoría y los productos son los tipos de informe. El reto de agregar la familia «alto contraste» equivale a nuestra evolución MIXTA (sección Evolución 1), mientras que agregar un producto nuevo es el caso PEDIDO (sección Evolución 2), la parte difícil del patrón.

2.3 Ventajas

A continuación, se presentan las principales ventajas que ofrece el patrón Abstract Factory en el diseño del sistema de procesamiento de informes. Estas se analizan desde la perspectiva de la separación de responsabilidades, el desacoplamiento, la consistencia en la creación de objetos y la capacidad de evolución de la solución, considerando tanto sus fundamentos teóricos como los resultados obtenidos durante su implementación.

- **Aísla las clases concretas** (Tema 3): `ProcesadorInformes` no contiene ningún `new` de una clase concreta: solo conoce `InformeFactory` e `Informe`.
- **Soporta nuevas familias sin modificar el cliente:** Añadir MIXTA costó 3 clases nuevas y 1 línea de registro; el procesador, la interfaz de fábrica y el enum no cambiaron (Ejercicio 3).
- **Promueve la consistencia** (Tema 3): una fábrica solo produce informes de su categoría, por lo que no pueden mezclarse productos de familias distintas.
- **Facilita el intercambio de familias** (Tema 3) y mejora la cohesión: cada informe encapsula su propio procesado (polimorfismo), de modo que puede probarse y evolucionar de forma aislada.
- **Vocabulario común y separación de responsabilidades:** Nombrar el patrón permite que el equipo se comunique con precisión (por ejemplo, «cada categoría es una fábrica concreta»), y la lógica de creación queda separada de la lógica de uso: el procesador no crea informes, solo los usa.

2.4 Inconvenientes

A pesar de los beneficios que aporta el patrón Abstract Factory, su aplicación también implica ciertos compromisos en términos de complejidad estructural, cantidad de código y flexibilidad para incorporar nuevos productos. A continuación, se examinan sus principales inconvenientes, considerando las características del problema planteado y las dificultades identificadas durante la evolución del sistema.

- **Más clases y más código:** Para el problema base el patrón requiere 15 archivos y 261 líneas de producción frente a 1 archivo de 35 líneas sin patrón. Con solo cuatro comportamientos triviales, el patrón es sobre-ingeniería; solo se justifica por la evolución prevista y porque cada informe real tendrá lógica propia.
- **Añadir un nuevo producto es costoso:** Como señala el Tema 3, para agregar productos hay que modificar la fábrica abstracta y todas las concretas. Es el punto débil que pone a prueba el tipo PEDIDO en el Ejercicio 3.
- **Indirección y decisión inicial:** Seguir el flujo exige navegar fábrica → producto; además el patrón no elimina la decisión inicial, solo la aísla: sigue haciendo falta traducir el nombre del archivo a una fábrica y una operación (FabricaRegistro y TipoInforme).
- **Supone familias homogéneas:** Asume que todas las fábricas ofrecen los mismos productos; PEDIDO existe solo en ENTRADA y rompe esa hipótesis.

2.5 Alternativas consideradas

Con el fin de seleccionar la solución de diseño más adecuada para el sistema de procesamiento de informes, se analizaron diferentes patrones de creación abordados en el aula y otras alternativas del catálogo GoF. La Tabla 2 presenta una comparación de sus intenciones, el grado de adecuación al problema y las razones que sustentan su elección o descarte. Este análisis permite justificar técnicamente la selección de Abstract Factory frente a las demás alternativas, considerando las dos dimensiones de clasificación de los informes, la coherencia entre familias de productos y las necesidades de evolución del sistema.

Tabla 2. Patrones de creación del aula frente al problema.

Patrón	Intención (según el aula)	Encaje	Justificación
Abstract Factory	Crear familias de productos relacionados sin especificar clases concretas (Tema 3).	**Elegido**	Dos dimensiones (categoría y tipo), familias coherentes y nuevas familias previstas (MIXTA).
Factory Method	Que las subclases decidan qué clase instanciar; crea **un solo producto** (Tema 4).	Parcial	Exigiría un creador por tipo y perdería la coherencia por categoría; Abstract Factory suele implementarse con él.
Builder	Construir paso a paso un objeto complejo con distintas representaciones (Tema 3).	No	Un informe no se ensambla por partes; no hay *director*.
Singleton	Una única instancia con punto de acceso conocido (Tema 4).	Complemento	No crea productos, pero es la forma habitual de implementar las fábricas concretas; se usa para ellas.
Prototype	Crear objetos clonando un prototipo existente (quinto patrón de creación del catálogo GoF; no figura en el temario, pero se explicó en clase).	No	No hay objetos prototipo que clonar: cada informe se crea a partir de su nombre.

Criterio de selección: Siguiendo la metodología vista en clase (especificar el problema, revisar el catálogo de 23 patrones y sus tres categorías, estudiar la estructura, nombrar los participantes e implementar), el patrón no se eligió por ser el más usado (Factory Method lo es), sino por ajustarse al problema: dos dimensiones y familias completas.

Conclusión del ejercicio: se mantiene *Abstract Factory*, complementado con *Singleton* para las fábricas concretas. Se asume el coste de la indirección y la debilidad frente a nuevos productos, que se analiza en el Ejercicio 3.

2.6 Reutilización de software en la solución

La asignatura entiende la reutilización como el uso de activos ya existentes y probados para evitar construir desde cero. Distingue la reutilización **oportunista** (se descubre sobre la marcha) de la **planificada** (el sistema se diseña para que sus elementos sean reutilizables), y *consumir* elementos de otros de *diseñar para reutilizar*. Esta solución es un caso de diseño para reutilizar:

- **Se reutiliza sin modificar:** Añadir la categoría MIXTA no modificó el procesador, la interfaz de fábrica ni las fábricas existentes; añadir AJUSTE (etapa 4b) no modificó ninguna línea de Java existente (capítulo 5).
- **Se reutiliza parcialmente un patrón cuando conviene:** Abstract Factory se complementa con Singleton (fábricas concretas), una interfaz derivada para PEDIDO y ServiceLoader, en lugar de aplicarlo de forma rígida.
- **Reutilizar a ciegas es un riesgo:** En clase se comentó el caso del cohete Ariane 5, cuyo módulo de la versión anterior se reutilizó sin adaptarlo ni probarlo adecuadamente para las nuevas condiciones y falló a los pocos segundos del despegue (Sommerville, 2016). Por eso cada etapa se verificó: la salida de la etapa 3 se comparó con la de la etapa 5 (idéntica) y la suite de 29 pruebas protege lo ya construido ante cada cambio.

3. EJERCICIO 2: DISEÑO E IMPLEMENTACIÓN

En este ejercicio se desarrolla el diseño estructural y la implementación del patrón Abstract Factory aplicado al sistema de procesamiento de informes. A partir de los requisitos establecidos, se define la arquitectura de clases mediante diagramas UML y se implementan en Java las interfaces, fábricas concretas y productos correspondientes a cada categoría y tipo de informe. Finalmente, se verifica el funcionamiento de la solución mediante pruebas de ejecución, evaluando la correcta creación de las familias de objetos y la separación de responsabilidades entre sus componentes.

3.1 Participantes del patrón

Para comprender la estructura de la solución implementada, es necesario identificar los participantes que conforman el patrón Abstract Factory y establecer su correspondencia con los componentes del sistema de procesamiento de informes. La Tabla 3 presenta esta relación, describiendo las responsabilidades de cada elemento y diferenciando los componentes propios del patrón de aquellos que cumplen funciones complementarias dentro de la arquitectura.

Tabla 3. Correspondencia entre los participantes del patrón (Tema 3) y el sistema.

Participante del patrón	Elemento en el sistema
FábricaAbstracta	InformeFactory: crearFactura(nombre), crearCompra(nombre)
FábricaConcreta (1..n)	EntradaFactory, SalidaFactory (implementadas como *Singleton*)
ProductoAbstracto (A, B)	Factura y Compra, ambas especializaciones de Informe
ProductoConcreto	FacturaEntrada, CompraEntrada, FacturaSalida, CompraSalida
Cliente	ProcesadorInformes: depende solo de InformeFactory e Informe
Apoyo (no pertenece al patrón)	FabricaRegistro, TipoInforme, InformeBase

3.2 Diagrama UML del sistema base

A continuación, se presenta el diagrama UML correspondiente al diseño base del sistema de procesamiento de informes, en el que se representan las interfaces, clases concretas y relaciones que conforman la implementación del patrón Abstract Factory. Este modelo permite visualizar la distribución de responsabilidades, la organización

de las familias de productos y las dependencias entre los componentes. Posteriormente, se incluye una lectura guiada del diagrama para explicar su estructura y el funcionamiento de sus principales relaciones.

Lectura del diagrama: (1) ProcesadorInformes depende únicamente de abstracciones (InformeFactory, Informe) y de los dos apoyos FabricaRegistro y TipoInforme. (2) Cada fábrica concreta (EntradaFactory, SalidaFactory) crea exactamente una familia: las flechas «usa» y «crea» no cruzan familias. (3) Los productos concretos realizan Factura o Compra, que a su vez especializan Informe; es la raíz de la jerarquía de productos y la que expone procesar().

3.3 Implementación en Java

La implementación del patrón de diseño Abstract Factory se desarrolla en el lenguaje de programación Java, utilizando sus mecanismos de interfaces, clases abstractas, herencia y polimorfismo para representar las familias de productos y las fábricas responsables de su creación. El propósito de esta implementación es trasladar el diseño conceptual definido mediante UML a una estructura de código que permita mantener la independencia entre el cliente y las implementaciones concretas, garantizando que cada categoría de informe disponga de su propia familia de productos. Para facilitar la organización, comprensión y mantenimiento del código fuente, el proyecto se estructura en paquetes de acuerdo con las responsabilidades de sus componentes, evitando concentrar toda la lógica en una única clase o módulo. Esta distribución permite diferenciar las abstracciones del modelo, las interfaces de creación, las implementaciones concretas de cada familia y los componentes encargados de coordinar la ejecución de la aplicación. La estructura se organiza bajo el directorio etapa1_base/src/informes, distribuida de la siguiente manera:

- Paquete modelo: contiene las abstracciones principales del dominio. La interfaz Informe define el contrato general de procesamiento, mientras que InformeBase proporciona una implementación común. Las clases Factura y Compra representan los productos abstractos que serán especializados por las distintas familias.

- Paquete fabrica: incorpora la interfaz InformeFactory, que establece los métodos de creación de los productos de cada familia, y TipoInforme, que proporciona el mecanismo de identificación de los tipos de informe admitidos por el sistema.

- Paquete entrada: implementa la familia concreta correspondiente a los informes de entrada mediante EntradaFactory, FacturaEntrada y CompraEntrada. Estos componentes encapsulan las particularidades de procesamiento de dicha categoría.

- Paquete salida: contiene la implementación de la familia de informes de salida, conformada por SalidaFactory, FacturaSalida y CompraSalida, manteniendo una estructura equivalente a la familia anterior, pero con comportamientos específicos de su categoría.

- Paquete app: reúne los componentes responsables de la coordinación y ejecución de la aplicación. Incluye FabricaRegistro, que permite localizar las fábricas concretas; ProcesadorInformes, que actúa como cliente del patrón; y Main, que constituye el punto de entrada para ejecutar y comprobar el funcionamiento del sistema.

Esta organización refleja la separación de responsabilidades propuesta por Abstract Factory: el cliente solicita la creación de los productos mediante una abstracción, sin necesitar conocer las clases concretas que los implementan. De esta manera, ProcesadorInformes puede trabajar con las distintas familias de informes a través de contratos comunes, mientras que cada fábrica concreta conserva la responsabilidad de proporcionar los productos correspondientes a su categoría. A continuación, muestro como el proyecto se organiza en paquetes por responsabilidad (etapa1_base/src/informes):

```
informes/
  |-- modelo/   Informe, InformeBase, Factura, Compra         (productos abstractos)
  |-- fabrica/  InformeFactory, TipoInforme                   (fábrica abstracta + apoyo)
  |-- entrada/  EntradaFactory, FacturaEntrada, CompraEntrada (familia ENTRADA)
  |-- salida/   SalidaFactory,  FacturaSalida,  CompraSalida  (familia SALIDA)
  -- app/      FabricaRegistro, ProcesadorInformes, Main     (cliente)
```

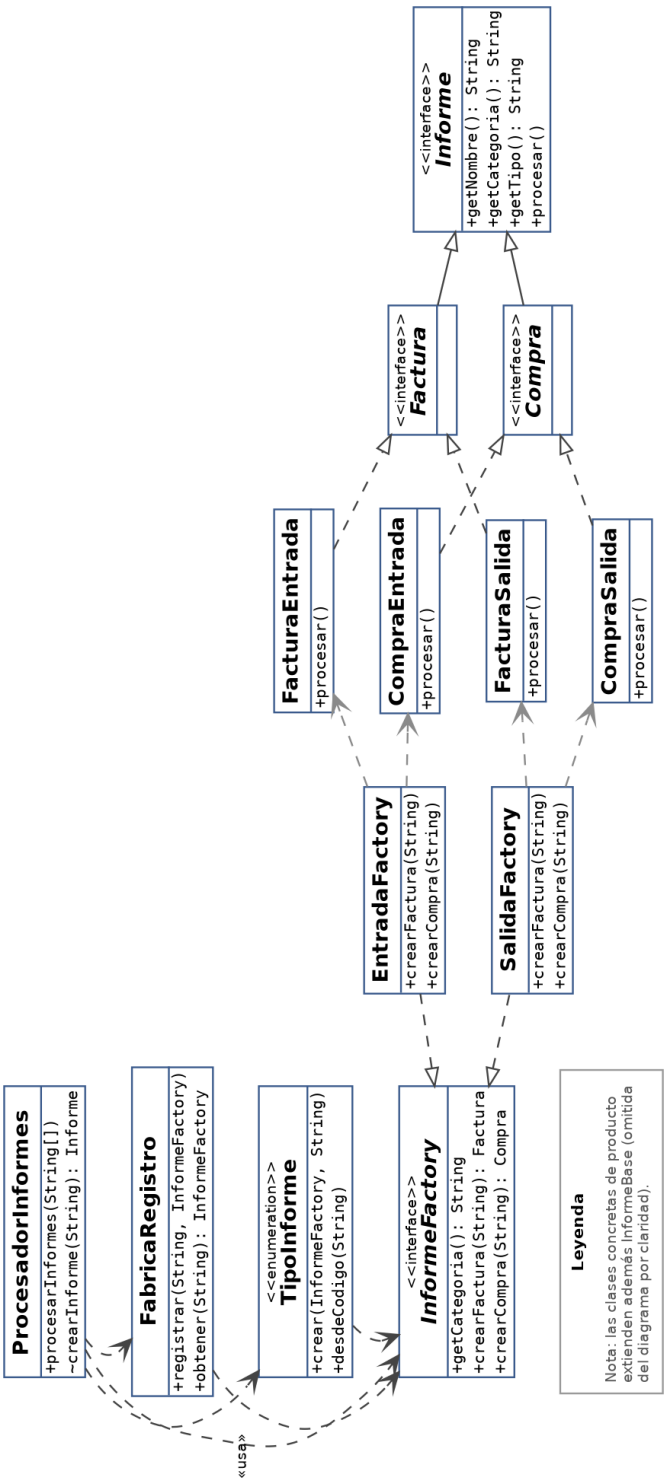


Figura 1. Diagrama de clases UML del sistema base con Abstract Factory (categorías ENTRADA y SALIDA; tipos FACTURA y COMPRA).

Fábrica abstracta: define qué productos puede crear cualquier categoría:

```
public interface InformeFactory {
    String getCategoria();
    Factura crearFactura(String nombre);
    Compra crearCompra(String nombre);
}
```

Fábrica concreta (SalidaFactory es análoga): crea la familia ENTRADA y es *Singleton*, tal como sugiere el Tema 3 para las fábricas concretas:

```
public final class EntradaFactory implements InformeFactory {
    private static final EntradaFactory INSTANCIA = new EntradaFactory();
    private EntradaFactory() { }
    public static EntradaFactory getInstance() { return INSTANCIA; }
    @Override public String getCategoria() { return "ENTRADA"; }
    @Override public Factura crearFactura(String nombre) { return new FacturaEntrada(nombre); }
    @Override public Compra crearCompra(String nombre) { return new CompraEntrada(nombre); }
}
```

Un producto concreto: cada informe encapsula su propio modo de procesado (FacturaEntrada; el resto sigue el mismo esquema):

```
public class FacturaEntrada extends InformeBase implements Factura {
    public FacturaEntrada(String nombre) { super(nombre); }
    @Override public String getCategoria() { return "ENTRADA"; }
    @Override public String getTipo() { return "FACTURA"; }
    @Override public void procesar() { trazar("Registrando factura recibida en cuentas por pagar"); }
}
```

Traducción del tipo a la operación de la fábrica: el enum evita un switch en el cliente: un tipo nuevo es una constante nueva.

```
public enum TipoInforme {
    FACTURA("FAC") {
        @Override
        public Informe crear(InformeFactory fabrica, String nombre) {
            return fabrica.crearFactura(nombre);
        }
    },
    COMPRA("COM") {
        @Override
        public Informe crear(InformeFactory fabrica, String nombre) {
            return fabrica.crearCompra(nombre);
        }
    };
    private final String codigo;
    TipoInforme(String codigo) {
        this.codigo = codigo;
    }
    public abstract Informe crear(InformeFactory fabrica, String nombre);
    public static TipoInforme desdeCodigo(String codigo) {
        for (TipoInforme t : values()) {
            if (t.codigo.equalsIgnoreCase(codigo)) {
                return t;
            }
        }
        throw new IllegalArgumentException("Tipo de informe desconocido: " + codigo);
    }
}
```

Cliente: no contiene ningún new de clases concretas ni condicionales por categoría o tipo:

```
public class ProcesadorInformes {
    public void procesarInformes(String[] nombresArchivo) {
        for (String nombre : nombresArchivo) {
            try {
                crearInforme(nombre).procesar();
            } catch (IllegalArgumentException | UnsupportedOperationException e) {
                System.out.println("[AVISO] " + nombre + " -> " + e.getMessage());
            }
        }
    }
        Informe crearInforme(String nombreArchivo) {
        String[] partes = nombreArchivo.split("_");
        if (partes.length < 3 || !nombreArchivo.toLowerCase().endsWith(".txt")) {
            throw new IllegalArgumentException("formato inválido (se esperaba CAT_TIPO_nombre.txt)");
        }
        InformeFactory fabrica = FabricaRegistro.obtener(partes[0]);
        TipoInforme tipo = TipoInforme.desdeCodigo(partes[1]);
        return tipo.crear(fabrica, nombreArchivo);
    }
}
```

Programa principal: del estilo pedido en el enunciado:

```
public class Main {
    public static void main(String[] args) {
        String[] reports = {
            "ENT_FAC_001.txt",
            "ENT_COM_001.txt",
            "SAL_FAC_001.txt",
            "SAL_COM_001.txt",
            // Casos no válidos: el sistema los reporta sin detenerse
            "XYZ_FAC_001.txt",   // categoría inexistente
            "SAL_PED_001.txt"    // tipo inexistente en esta etapa
        };
        System.out.println("=== ETAPA 1: sistema base (ENTRADA / SALIDA x FACTURA / COMPRA) ===");
        new ProcesadorInformes().procesarInformes(reports);
    }
}
```

Se omiten las líneas package e import y la clase FabricaRegistro (un Map<String, InformeFactory> con dos llamadas a registrar). El código completo está en el repositorio indicado en la ficha del documento.

3.4 Decisiones de diseño

La implementación del patrón Abstract Factory requiere establecer decisiones de diseño que permitan mantener una arquitectura organizada, coherente y preparada para su evolución. Estas decisiones no se limitan a definir la distribución de las clases y los paquetes, sino que también determinan cómo se gestionan las solicitudes de creación, cómo se reutiliza el comportamiento común, cómo se controlan los errores y qué criterios de compatibilidad se consideran durante el desarrollo. En este sentido, se adoptaron cinco decisiones principales, orientadas a reducir el acoplamiento entre los componentes, evitar la duplicación de código y garantizar un comportamiento predecible durante la ejecución del sistema:

- **FabricaRegistro + TipoInforme** concentran toda la traducción «texto → fábrica/operación». El resto del sistema trabaja con tipos, no con cadenas.
- **Fábricas concretas como *Singleton*** con inicialización inmediata (segura ante concurrencia): son objetos sin estado, así que una instancia basta.
- **InformeBase** evita repetir el nombre y la traza de salida; cada producto solo declara su categoría, su tipo y su acción específica.
- **Errores controlados:** una categoría o tipo desconocido, o un nombre con formato inválido, se reportan con [AVISO] sin detener el procesado de los demás archivos.
- **Compatibilidad:** el código usa solo características de Java 8 (sin *pattern matching*), y se probó con JDK 17 y 21.

3.5 Resultados de la ejecución

La Figura 2 muestra la salida real del programa. Los cuatro informes válidos se procesan mediante una llamada polimórfica a procesar(), indicando categoría y tipo; los dos últimos (categoría inexistente y tipo aún inexistente) se informan sin interrumpir la ejecución.

```
$ ./compilar_ejecutar.sh etapa1_base
=== ETAPA 1: sistema base (ENTRADA / SALIDA x FACTURA / COMPRA) ===
Procesando informe de ENTRADA de tipo FACTURA  [ENT_FAC_001.txt]
      -> Registrando factura recibida en cuentas por pagar
Procesando informe de ENTRADA de tipo COMPRA  [ENT_COM_001.txt]
      -> Validando la compra recibida contra el inventario
Procesando informe de SALIDA de tipo FACTURA  [SAL_FAC_001.txt]
      -> Emitiendo factura al cliente y registrándola en cuentas por cobrar
Procesando informe de SALIDA de tipo COMPRA  [SAL_COM_001.txt]
      -> Generando la orden de compra dirigida al proveedor
[AVISO] XYZ_FAC_001.txt -> Categoría de informe desconocida: XYZ
[AVISO] SAL_PED_001.txt -> Tipo de informe desconocido: PED
```

Figura 2. Ejecución del sistema base (etapa1_base).

4. EJERCICIO 3: EVOLUCIÓN DEL SISTEMA

Para analizar la utilidad del patrón se evoluciona el sistema en dos pasos, conservando cada etapa como proyecto independiente y compilable (etapa2_mixta, etapa3_pedido). Así puede compararse el código de cada etapa con el de la anterior.

4.1 Evolución 1: nueva categoría MIXTA

En esta primera evolución se incorpora la categoría MIXTA al sistema de procesamiento de informes, con el propósito de evaluar la capacidad del patrón Abstract Factory para admitir nuevas familias de productos sin alterar la estructura de las implementaciones existentes. Esta categoría representa una nueva familia que debe mantener la misma organización y los contratos definidos en el diseño base, proporcionando sus propias implementaciones para los tipos de informe FACTURA y COMPRA. Para llevar a cabo esta extensión, se implementa la fábrica concreta MixtaFactory, encargada de crear los productos FacturaMixta y CompraMixta, los cuales representan los comportamientos específicos de esta nueva categoría. Adicionalmente, se incorpora su registro en el mecanismo de identificación de fábricas, mediante una modificación puntual que permite al sistema reconocer y utilizar la nueva familia. Un aspecto relevante de esta evolución es que no requiere modificar componentes centrales como ProcesadorInformes, la interfaz InformeFactory, el enumerado TipoInforme ni las fábricas concretas previamente implementadas. Esto permite comprobar cómo la arquitectura facilita la incorporación de nuevas categorías mediante la extensión de sus componentes, conservando las abstracciones y responsabilidades establecidas desde la primera etapa. A continuación, se presentan los cambios realizados en el código fuente de la etapa etapa2_mixta, junto con los componentes nuevos y la modificación puntual necesaria para integrar esta familia al sistema.

Cambios necesarios: Una categoría nueva es una *familia* nueva, el caso en que el patrón está diseñado para ayudar:

- Crear MixtaFactory (implementa InformeFactory) y los dos productos FacturaMixta y CompraMixta.
- Dar de alta la fábrica en el registro: **una línea** (más su import).

No se modifican ProcesadorInformes, InformeFactory, TipoInforme ni las fábricas existentes. Código nuevo (etapa2_mixta):

```
public final class MixtaFactory implements InformeFactory {
    private static final MixtaFactory INSTANCIA = new MixtaFactory();
    private MixtaFactory() { }
    public static MixtaFactory getInstance() { return INSTANCIA; }
    @Override public String getCategoria() { return "MIXTA"; }
    @Override public Factura crearFactura(String nombre) { return new FacturaMixta(nombre); }
    @Override public Compra crearCompra(String nombre) { return new CompraMixta(nombre); }
}
```

```
// FabricaRegistro.java — único cambio en código existente (2 líneas)
+ import informes.mixta.MixtaFactory;
  ...
    registrar("ENT", EntradaFactory.getInstance());
    registrar("SAL", SalidaFactory.getInstance());
+   registrar("MIX", MixtaFactory.getInstance());
```

La Figura 3 (página siguiente) amplía el diagrama anterior: en verde, los elementos nuevos; en ámbar, los modificados.

4.1.1 Diagrama UML ampliado: categoría MIXTA

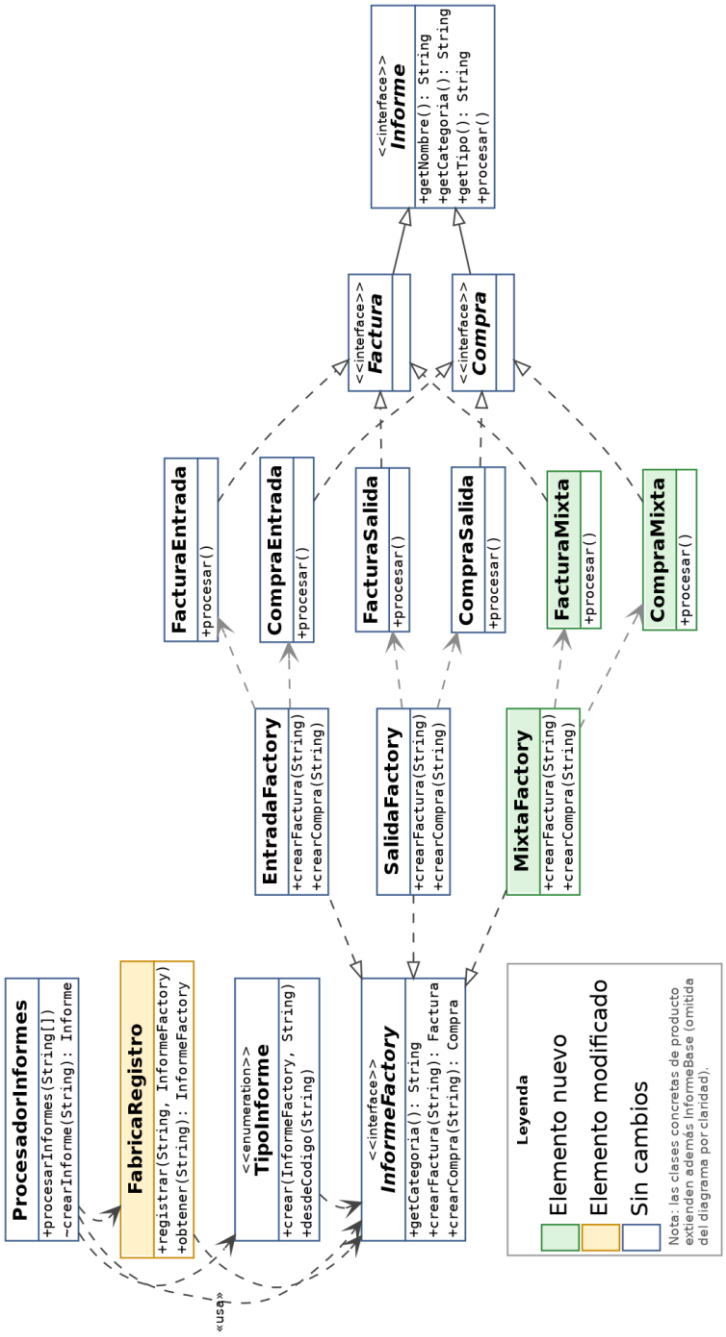


Figura 3. Diagrama UML tras añadir la categoría MIXTA (verde: nuevo; ámbar: modificado respecto a la Figura 1).

Lectura: MixtaFactory, FacturaMixta y CompraMixta forman una familia completa que se acopla al sistema únicamente a través de InformeFactory. Solo FabricaRegistro cambia, y únicamente para conocer la nueva fábrica.

Resultado: Los informes MIX_FAC_001.txt y MIX_COM_001.txt se procesan correctamente y los anteriores no se alteran (Figura 4):

```
$ ./compilar_ejecutar.sh etapa2_mixta
=== ETAPA 2: se añade la categoría MIXTA ===
Procesando informe de ENTRADA de tipo FACTURA  [ENT_FAC_001.txt]
      -> Registrando factura recibida en cuentas por pagar
Procesando informe de ENTRADA de tipo COMPRA  [ENT_COM_001.txt]
      -> Validando la compra recibida contra el inventario
Procesando informe de SALIDA de tipo FACTURA  [SAL_FAC_001.txt]
      -> Emitiendo factura al cliente y registrándola en cuentas por cobrar
Procesando informe de SALIDA de tipo COMPRA  [SAL_COM_001.txt]
      -> Generando la orden de compra dirigida al proveedor
Procesando informe de MIXTA de tipo FACTURA  [MIX_FAC_001.txt]
      -> Conciliando facturas de entrada y de salida en un único informe
Procesando informe de MIXTA de tipo COMPRA  [MIX_COM_001.txt]
      -> Consolidando compras recibidas y emitidas
[AVISO] XYZ_FAC_001.txt -> Categoría de informe desconocida: XYZ
[AVISO] SAL_PED_001.txt -> Tipo de informe desconocido: PED
```

Figura 4. Ejecución tras añadir la categoría MIXTA (etapa2_mixta).

4.2 Evolución 2: nuevo tipo PEDIDO en la categoría ENTRADA

En esta segunda evolución se incorpora el tipo de informe PEDIDO, una nueva funcionalidad que debe estar disponible exclusivamente para la categoría ENTRADA. Esta modificación plantea un escenario diferente al abordado en la evolución anterior, puesto que ya no se trata de incorporar una familia completa de productos, sino de ampliar una familia existente mediante un nuevo producto y su correspondiente operación de creación.

Aquí se evidencia una de las principales debilidades del patrón Abstract Factory en su estructura convencional. Cuando se incorpora un nuevo producto, es necesario ampliar el contrato de la fábrica abstracta mediante la declaración de un nuevo método, en este caso crearPedido(). Sin embargo, esta modificación también repercute en todas las fábricas concretas que implementan dicha interfaz, incluso en aquellas categorías que no tienen previsto ofrecer la funcionalidad de pedidos. Esto puede introducir implementaciones innecesarias, métodos que no representan una responsabilidad real para determinadas fábricas o comportamientos artificiales destinados únicamente a satisfacer el contrato común.

En el problema planteado, esta situación resulta especialmente relevante porque las categorías ENTRADA, SALIDA y MIXTA no comparten necesariamente las mismas capacidades funcionales. Mientras que ENTRADA debe permitir la creación de informes de tipo PEDIDO, las demás categorías no requieren esta operación. Por consiguiente, obligar a todas las fábricas a implementar el mismo método podría afectar la coherencia de sus contratos y generar dependencias que no corresponden a sus responsabilidades.

Ante esta situación, se evaluaron tres alternativas de diseño para incorporar PEDIDO, considerando el impacto de cada una sobre las interfaces, las fábricas concretas, el mantenimiento del código y los principios de diseño orientado a objetos. El propósito es identificar una solución que permita extender la funcionalidad de ENTRADA sin imponer obligaciones innecesarias a las demás categorías y, al mismo tiempo, reconocer las limitaciones que introduce cada alternativa frente a la estructura original de Abstract Factory. Se evaluaron tres opciones:

Tabla 4. Opciones para incorporar el tipo PEDIDO solo a ENTRADA.

Opción	Código que cambia	Consecuencias	Decisión
A. Añadir crearPedido() a InformeFactory	Interfaz + 3 fábricas concretas	SALIDA y MIXTA deberían implementar un método sin sentido (excepción o null). Modifica código estable y viola la Segregación de Interfaces.	Descartada

Opción	Código que cambia	Consecuencias	Decisión
B. Método default que lanza UnsupportedOperationException	Interfaz + EntradaFactory	Menos cambios, pero el contrato común anuncia una capacidad que casi nadie tiene; el error aparece solo en ejecución.	Válida, no elegida
C. Interfaz derivada InformeFactoryConPedido	Interfaz nueva + EntradaFactory	El contrato expresa qué categorías admiten pedidos. SalidaFactory, MixtaFactory e InformeFactory no cambian. Coste: una comprobación instanceof en un único punto.	**Elegida**

Se elige la opción **C** por aplicar el principio de Segregación de Interfaces (Martin, 2002): limita el impacto a una fábrica concreta y a una constante del enum, y deja abierta la ampliación (si SALIDA admitiera pedidos en el futuro, bastaría con que SalidaFactory implemente la misma interfaz).

Cambios necesarios (etapa3_pedido): crear Pedido, PedidoEntrada e InformeFactoryConPedido; hacer que EntradaFactory implemente la interfaz derivada; y añadir la constante PEDIDO al enum.

```
/
 * Extensión de la fábrica abstracta para las categorías que SÍ manejan pedidos.
 * Se usa una interfaz derivada (Segregación de Interfaces) en lugar de añadir
 * crearPedido() a InformeFactory: así SALIDA y MIXTA no se ven obligadas a
 * implementar un producto que no les corresponde y su código no cambia.
 */
public interface InformeFactoryConPedido extends InformeFactory {
    Pedido crearPedido(String nombre);
}
```

Cambios en código existente:

```
// EntradaFactory.java
- public final class EntradaFactory implements InformeFactory {
+ public final class EntradaFactory implements InformeFactoryConPedido {
  ...
+   @Override public Pedido crearPedido(String nombre) { return new PedidoEntrada(nombre); }

// TipoInforme.java — nueva constante del enum
+     PEDIDO("PED") {
+         @Override
+         public Informe crear(InformeFactory fabrica, String nombre) {
+             if (fabrica instanceof InformeFactoryConPedido) {
+                 return ((InformeFactoryConPedido) fabrica).crearPedido(nombre);
+             }
+             throw new UnsupportedOperationException("la categoría " + fabrica.getCategoria()
+                     + " no admite informes de tipo PEDIDO");
+         }
+     };
```

La Figura 5 (página siguiente) muestra el diagrama final.

4.2.1 Diagrama UML ampliado: tipo PEDIDO

Lectura: InformeFactoryConPedido extiende InformeFactory y solo la implementa EntradaFactory. SalidaFactory y MixtaFactory quedan intactas. El único punto que conoce la existencia de la interfaz derivada es la constante PEDIDO de TipoInforme.

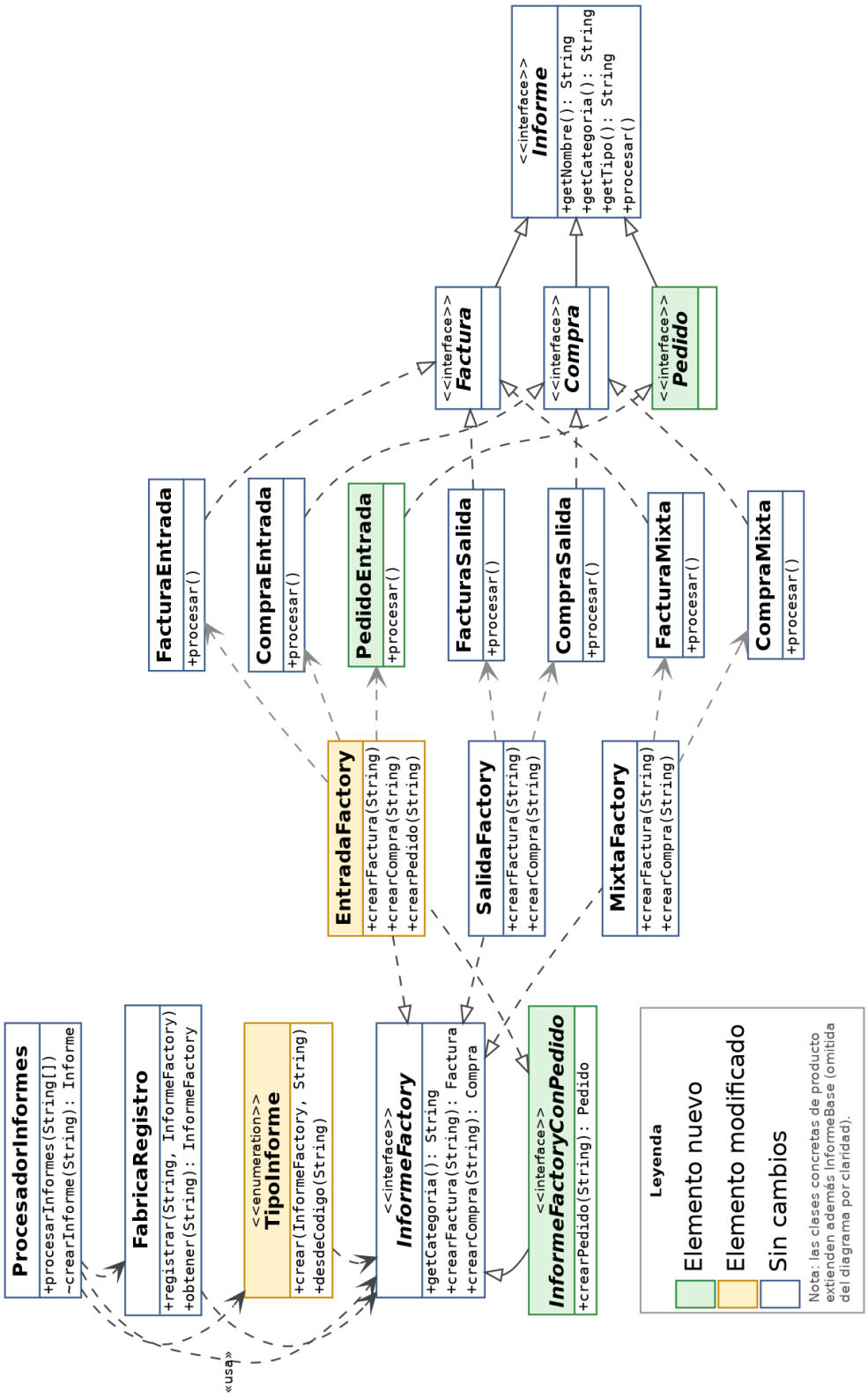


Figura 5. Diagrama UML final con el tipo PEDIDO en ENTRADA (verde: nuevo respecto a la Figura 3; ámbar: modificado).

Resultado: ENT_PED_001.txt se procesa con la lógica propia de pedidos. SAL_PED_001.txt y MIX_PED_001.txt —tipos no admitidos por esas categorías— se rechazan con un aviso claro en lugar de fallar o procesarse mal (Figura 6):

```
$ ./compilar_ejecutar.sh etapa3_pedido
=== ETAPA 3: se añade el tipo PEDIDO a la categoría ENTRADA ===
Procesando informe de ENTRADA de tipo FACTURA  [ENT_FAC_001.txt]
      -> Registrando factura recibida en cuentas por pagar
Procesando informe de ENTRADA de tipo COMPRA  [ENT_COM_001.txt]
      -> Validando la compra recibida contra el inventario
Procesando informe de SALIDA de tipo FACTURA  [SAL_FAC_001.txt]
      -> Emitiendo factura al cliente y registrándola en cuentas por cobrar
Procesando informe de SALIDA de tipo COMPRA  [SAL_COM_001.txt]
      -> Generando la orden de compra dirigida al proveedor
Procesando informe de MIXTA de tipo FACTURA  [MIX_FAC_001.txt]
      -> Conciliando facturas de entrada y de salida en un único informe
Procesando informe de MIXTA de tipo COMPRA  [MIX_COM_001.txt]
      -> Consolidando compras recibidas y emitidas
Procesando informe de ENTRADA de tipo PEDIDO  [ENT_PED_001.txt]
      -> Registrando el pedido recibido y reservando existencias
[AVISO] XYZ_FAC_001.txt -> Categoría de informe desconocida: XYZ
[AVISO] SAL_PED_001.txt -> la categoría SALIDA no admite informes de tipo PEDIDO
[AVISO] MIX_PED_001.txt -> la categoría MIXTA no admite informes de tipo PEDIDO
```

Figura 6. Ejecución del sistema final (etapa3_pedido).

4.3 Análisis del impacto de las evoluciones

La incorporación de nuevas funcionalidades permite evaluar la capacidad de evolución de una arquitectura de software y determinar el impacto que los cambios generan sobre su estructura interna. En este ejercicio, se analiza el comportamiento de la implementación basada en el patrón Abstract Factory durante dos evoluciones sucesivas: la incorporación de la categoría MIXTA en la segunda etapa y la incorporación del tipo PEDIDO en la tercera etapa.

Para realizar esta evaluación, se consideran medidas relacionadas con el tamaño del código fuente, la cantidad de archivos nuevos, los archivos de producción existentes que requieren modificaciones y el número de líneas añadidas o eliminadas. Asimismo, se identifica si las evoluciones afectan componentes centrales como ProcesadorInformes e InformeFactory, con el propósito de determinar si la arquitectura conserva la separación de responsabilidades y el desacoplamiento establecidos en el diseño inicial.

La Tabla 5 presenta los resultados obtenidos mediante la comparación real entre las tres etapas de implementación con Abstract Factory. Adicionalmente, incorpora una referencia de la versión sin patrón, específicamente el crecimiento de las líneas de código del procesador, para contrastar el esfuerzo de adaptación de ambas alternativas.

Este análisis permite observar cómo evoluciona la estructura del sistema a medida que se incorporan nuevos requisitos y qué modificaciones deben realizarse para soportarlos. De esta manera, se busca establecer si Abstract Factory facilita la extensión de las familias de productos, reduce la intervención sobre los componentes existentes y contribuye a mantener una arquitectura más estable y mantenible frente a los cambios.

Tabla 5. Coste de cada evolución con Abstract Factory (comparación real entre etapas) frente a la versión sin patrón.

Medida	Etapa 1 (base)	Etapa 2 (+ MIXTA)	Etapa 3 (+ PEDIDO)
Archivos .java (incluye Main)	15	18	21
Líneas de código de producción (sin Main)	261	306	348
Archivos nuevos	—	3	3
Archivos de producción existentes modificados	—	1 (FabricaRegistro, +2 líneas)	2 (EntradaFactory, TipoInforme)
Líneas añadidas / eliminadas en producción	—	+33 / −0	+38 / −2

Medida	Etapa 1 (base)	Etapa 2 (+ MIXTA)	Etapa 3 (+ PEDIDO)
Cliente (ProcesadorInformes) e InformeFactory modificados	—	No	No
Sin patrón: líneas del procesador	35	—	46 (tras ambas evoluciones)

Interpretación:

Los resultados presentados permiten identificar cómo el patrón Abstract Factory influye en el esfuerzo de mantenimiento y evolución del sistema. Al comparar las modificaciones realizadas en cada etapa, se evidencian diferencias importantes entre la incorporación de una nueva categoría y la incorporación de un nuevo tipo de informe, así como las ventajas y compromisos que implica mantener esta arquitectura.

A partir de los datos obtenidos, se establecen las siguientes observaciones:

- **MIXTA es el caso favorable:** Una familia nueva se incorpora añadiendo clases y una línea de registro; ningún código ya probado se reescribe. Sin patrón, la misma evolución obliga a editar el método central.
- **PEDIDO es el caso desfavorable:** Tal como anticipa el Tema 3. Aun así, con una interfaz derivada el impacto se limitó a dos archivos existentes (una fábrica concreta y un enum), y el contrato de las demás fábricas siguió siendo honesto.
- **Contrapartida honesta:** En tamaño absoluto el patrón genera más código que la versión sin patrón (348 frente a 46 líneas). Su ventaja no es escribir menos, sino **dónde** ocurre el crecimiento: con el patrón crece de forma localizada y aditiva (clases nuevas), mientras que sin él crece de forma concentrada y multiplicativa dentro de un método que cada vez contiene más lógica.
- **Limitación pendiente:** La comprobación instanceof en PEDIDO desplaza un error de compilación a un aviso en ejecución. Si todas las categorías llegaran a soportar pedidos, lo correcto sería fusionar crearPedido() en InformeFactory.

5. DESARROLLO ADICIONAL: MÁS ALLÁ DEL ENUNCIADO

El enunciado pide analizar la utilidad del patrón a partir de dos evoluciones. Este capítulo va más lejos con cuatro aportes, cada uno con evidencia medible y reproducible en el repositorio: (1) eliminar el acoplamiento que la solución del Ejercicio 2 todavía conservaba; (2) **verificar automáticamente** el diseño, incluida su arquitectura; (3) **cuantificar a escala** cuánto cuesta evolucionar, con un modelo calibrado contra mediciones reales; y (4) **contrastar el patrón con una alternativa** implementada, para decidir con criterio cuándo conviene.

Tabla 6. Mapa del desarrollo adicional y su evidencia.

Entregable	Qué aporta	Evidencia medible
etapa4_servicios	Categorías descubiertas con ServiceLoader (arquitectura de *plugins*).	Imports de clases concretas en app/: de 3 a 0.
etapa4b_ajuste	Nueva categoría AJUSTE añadida solo con archivos nuevos.	0 líneas de Java existente modificadas.
PruebasSistema	Suite autocontenida con una prueba de arquitectura.	29 de 29 correctas; una violación inyectada produce 1 fallo.
experimento/escala.py	Modelo de costes de evolución para tres diseños.	6 puntos de calibración coinciden con lo medido.
etapa5_registro	Alternativa sin Abstract Factory.	Salida idéntica a la etapa 3 (mismo hash) con 14 archivos y 211 líneas.

5.1 Etapa 4: categorías descubiertas con ServiceLoader

Problema detectado: Tras el Ejercicio 3, ProcesadorInformes ya no conocía ninguna clase concreta, pero FabricaRegistro sí: importaba EntradaFactory, SalidaFactory y MixtaFactory. Es decir, el paquete app seguía

dependiendo de todos los paquetes de categoría, y cada categoría nueva exigía editar código de ese paquete (una línea más su import). El acoplamiento no estaba en el cliente, sino desplazado al registro.

Solución: Se introduce un contrato de *plugin*, CategoriaInforme (codigo() y fabrica()), y cada categoría se anuncia a sí misma en un descriptor META-INF/services. FabricaRegistro pasa a descubrirlas con java.util.ServiceLoader, de modo que el núcleo no nombra ninguna categoría:

```
// FabricaRegistro.java (etapa 4): ya no importa ninguna fábrica concreta
static {
    for (CategoriaInforme categoria : ServiceLoader.load(CategoriaInforme.class)) {
        registrar(categoria.codigo(), categoria.fabrica());
    }
}

// META-INF/services/informes.fabrica.CategoriaInforme
informes.entrada.EntradaCategoria
informes.salida.SalidaCategoria
informes.mixta.MixtaCategoria
```

Prueba de la tesis (etapa 4b): Se añade una cuarta categoría, AJUSTE (AJU_FAC_001.txt, AJU_COM_001.txt), creando únicamente archivos nuevos y una línea en el descriptor. El Main de la etapa 4 ya incluía esos dos nombres, por lo que **no se modificó**: en la etapa 4 se reportan como categoría desconocida y en la 4b se procesan (Figura 7). La Figura 8 muestra la medición del cambio.

Cómo leer las Figuras 7 y 8: La salida de la Figura 7 corresponde al mismo Main de la etapa 4, ahora con la categoría AJUSTE disponible:

- Las líneas de ENTRADA (incluido PEDIDO), SALIDA y MIXTA son las mismas que en las etapas anteriores: añadir AJUSTE no alteró el comportamiento existente.
- AJU_FAC_001.txt y AJU_COM_001.txt se procesan con la lógica propia de la nueva familia («Aplicando ajustes contables sobre facturas ya emitidas» y «Corrigiendo compras ya registradas»).
- Los tres avisos finales se mantienen: la categoría XYZ sigue sin existir y SALIDA y MIXTA siguen rechazando PEDIDO, de modo que el tratamiento de errores tampoco cambió.
- La Figura 8 confirma el coste: diff solo detecta el paquete nuevo informes.ajuste y una línea añadida al descriptor META-INF/services (informes.ajuste.AjusteCategoria); las líneas .java existentes modificadas son 0.

Lectura crítica: La principal mejora introducida mediante ServiceLoader no debe interpretarse como una simple reducción de líneas de código, puesto que la instrucción que anteriormente permitía registrar las fábricas de manera explícita no desaparece por completo, sino que se traslada desde el código fuente hacia un descriptor de configuración. Por tanto, el beneficio obtenido no radica necesariamente en escribir menos código, sino en modificar la forma en que el sistema descubre e incorpora sus implementaciones concretas.

Desde una perspectiva arquitectónica, este cambio representa una mejora significativa en la inversión de dependencias. En la implementación anterior, el registro central debía conocer las categorías disponibles y mantener referencias directas a sus fábricas concretas. Esto generaba una dependencia del núcleo de la aplicación respecto de las implementaciones específicas, de manera que la incorporación de una nueva categoría exigía intervenir en el componente encargado del registro. Con ServiceLoader, el núcleo deja de depender directamente de las categorías y pasa a depender únicamente del contrato común definido por la interfaz de servicio. Las implementaciones concretas, por su parte, declaran su disponibilidad mediante el mecanismo de descubrimiento proporcionado por Java.

Esta separación permite que nuevas fábricas puedan incorporarse sin modificar el código fuente del núcleo, siempre que respeten el contrato establecido y se encuentren correctamente declaradas en el descriptor de servicios. Incluso es posible distribuir estas implementaciones en archivos .jar independientes, facilitando una arquitectura extensible en la que las categorías puedan desarrollarse, empaquetarse y desplegarse de manera separada. En consecuencia, el mecanismo favorece la modularidad y reduce la necesidad de recompilar el núcleo cuando se incorporan nuevas implementaciones, siempre que se mantengan la compatibilidad de las interfaces y las condiciones de ejecución correspondientes.

Sin embargo, esta decisión también introduce una nueva responsabilidad: la configuración del descubrimiento de servicios. Un error en el descriptor, como un nombre de clase incorrecto, una implementación inexistente o una clase que no cumple adecuadamente el contrato esperado, puede provocar un fallo durante la ejecución mediante una excepción ServiceConfigurationError. A diferencia de una referencia directa dentro del código fuente, algunos de estos problemas no se detectan durante la compilación, puesto que la resolución de las implementaciones depende del mecanismo de descubrimiento en tiempo de ejecución.

Por esta razón, la incorporación de ServiceLoader no debe considerarse una eliminación absoluta de los riesgos de acoplamiento o configuración, sino una redistribución de las responsabilidades y dependencias del sistema. La flexibilidad obtenida exige acompañar el mecanismo con pruebas que permitan verificar el descubrimiento de las fábricas, la validez de sus implementaciones, la ausencia de duplicidades y el comportamiento esperado cuando una categoría no se encuentra disponible. Asimismo, resulta conveniente comprobar que la incorporación de una nueva implementación no exige modificar ni recompilar el núcleo de la aplicación.

En definitiva, el aporte de ServiceLoader es principalmente arquitectónico: transforma un registro explícito y centralizado en un mecanismo de descubrimiento basado en contratos, favoreciendo la extensibilidad y la independencia de las implementaciones concretas. No obstante, esta flexibilidad debe estar respaldada por mecanismos de validación y pruebas automatizadas que permitan detectar oportunamente los errores introducidos en la configuración o en las implementaciones. Precisamente, esta necesidad justifica el siguiente aporte del ejercicio: la incorporación de pruebas orientadas a verificar no solo el funcionamiento de los productos, sino también el cumplimiento de las restricciones arquitectónicas definidas para el sistema.

```
$ ./compilar_ejecutar.sh etapa4b_ajuste
=== ETAPA 4: categorías descubiertas con ServiceLoader ===
Procesando informe de ENTRADA de tipo FACTURA  [ENT_FAC_001.txt]
      -> Registrando factura recibida en cuentas por pagar
Procesando informe de ENTRADA de tipo COMPRA  [ENT_COM_001.txt]
      -> Validando la compra recibida contra el inventario
Procesando informe de SALIDA de tipo FACTURA  [SAL_FAC_001.txt]
      -> Emitiendo factura al cliente y registrándola en cuentas por cobrar
Procesando informe de SALIDA de tipo COMPRA  [SAL_COM_001.txt]
      -> Generando la orden de compra dirigida al proveedor
Procesando informe de MIXTA de tipo FACTURA  [MIX_FAC_001.txt]
      -> Conciliando facturas de entrada y de salida en un único informe
Procesando informe de MIXTA de tipo COMPRA  [MIX_COM_001.txt]
      -> Consolidando compras recibidas y emitidas
Procesando informe de ENTRADA de tipo PEDIDO  [ENT_PED_001.txt]
      -> Registrando el pedido recibido y reservando existencias
Procesando informe de AJUSTE de tipo FACTURA  [AJU_FAC_001.txt]
      -> Aplicando ajustes contables sobre facturas ya emitidas
Procesando informe de AJUSTE de tipo COMPRA  [AJU_COM_001.txt]
      -> Corrigiendo compras ya registradas
[AVISO] XYZ_FAC_001.txt -> Categoría de informe desconocida: XYZ
[AVISO] SAL_PED_001.txt -> la categoría SALIDA no admite informes de tipo PEDIDO
[AVISO] MIX_PED_001.txt -> la categoría MIXTA no admite informes de tipo PEDIDO
```

Figura 7. Ejecución de la etapa 4b: AJUSTE se procesa sin tocar el Main ni el núcleo.

```
$ diff -rq etapa4_servicios/src etapa4b_ajuste/src
Only in etapa4b_ajuste/src/informes: ajuste
Files etapa4_servicios/src/META-INF/services/informes.fabrica.CategoriaInforme and etapa4b_ajuste/src/META-
INF/services/informes.fabrica.CategoriaInforme differ

$ diff etapa4_servicios/src/META-INF/services/informes.fabrica.CategoriaInforme etapa4b_ajuste/src/META-
INF/services/informes.fabrica.CategoriaInforme
3a4
> informes.ajuste.AjusteCategoria

$ diff -rq etapa4_servicios/src etapa4b_ajuste/src | grep -c '\.java .*differ'   # .java existentes
modificados
0
```

Figura 8. Medición de los cambios entre la etapa 4 y la 4b: 0 archivos .java existentes modificados.

5.2 Pruebas automatizadas, incluida una prueba de arquitectura

Se desarrolló PruebasSistema, una suite sin dependencias externas (no requiere JUnit ni descargas) que se ejecuta con ./probar.sh etapa4b_ajuste. Sus pruebas se **derivan del registro**, no de una lista fija: si mañana se añade una categoría, se prueba sola. La prueba P5 es la más ambiciosa, porque verifica una propiedad de diseño leyendo el código fuente del cliente:

Tabla 7. Qué protege cada grupo de pruebas (29 comprobaciones en la etapa 4b).

Grupo	Comprobación	Riesgo que evita	Casos
P1	Cada fábrica crea solo informes de su categoría y tipo correctos.	Mezclar productos de familias distintas.	12
P2	Formatos y códigos inválidos lanzan la excepción esperada.	Entradas corruptas que pasan sin aviso.	4
P3	PEDIDO se crea en ENTRADA y se rechaza en las demás.	Que la interfaz derivada pierda su propósito.	4
P4	Las fábricas concretas son *Singleton*.	Instancias duplicadas con estado inconsistente.	4
P5	**Arquitectura:** el cliente no importa categorías concretas, no usa new de productos ni compara cadenas con equals().	Que el acoplamiento regrese con una edición «cómoda».	5

Para comprobar que la prueba de arquitectura no es decorativa, se **inyectó una violación**: se añadió a ProcesadorInformes un import informes.entrada.EntradaFactory. La suite pasó de 29 correctas a 28 correctas y **1 fallo**, señalando exactamente esa regla. Una prueba que nunca puede fallar no protege nada; esta sí falla cuando debe.

```
$ ./probar.sh etapa4b_ajuste
== P1. Coherencia: cada fábrica solo produce informes de su categoría ==
  [OK]    ENT_FAC_001.txt -> ENTRADA/FACTURA
  [OK]    ENT_COM_001.txt -> ENTRADA/COMPRA
  [OK]    ENT_PED_001.txt -> ENTRADA/PEDIDO
  [OK]    SAL_FAC_001.txt -> SALIDA/FACTURA
  [OK]    SAL_COM_001.txt -> SALIDA/COMPRA
  [OK]    SAL_PED_001.txt -> rechazado: la categoría no admite ese tipo
  [OK]    MIX_FAC_001.txt -> MIXTA/FACTURA
  [OK]    MIX_COM_001.txt -> MIXTA/COMPRA
  [OK]    MIX_PED_001.txt -> rechazado: la categoría no admite ese tipo
  [OK]    AJU_FAC_001.txt -> AJUSTE/FACTURA
  [OK]    AJU_COM_001.txt -> AJUSTE/COMPRA
  [OK]    AJU_PED_001.txt -> rechazado: la categoría no admite ese tipo
== P2. Entradas inválidas se rechazan con la excepción adecuada ==
  [OK]    categoría inexistente (XYZ_FAC_001.txt)
  [OK]    tipo inexistente (ENT_ZZZ_001.txt)
  [OK]    formato sin tercer campo (ENT_FAC.txt)
  [OK]    extensión incorrecta (ENT_FAC_001.pdf)
== P3. PEDIDO existe únicamente en ENTRADA ==
  [OK]    ENT_PED_001.txt se crea
  [OK]    SAL_PED_001.txt se rechaza
  [OK]    MIX_PED_001.txt se rechaza
  [OK]    AJU_PED_001.txt se rechaza
== P4. Las fábricas concretas son Singleton ==
  [OK]    ENT: obtener() devuelve siempre la misma instancia
  [OK]    SAL: obtener() devuelve siempre la misma instancia
  [OK]    MIX: obtener() devuelve siempre la misma instancia
  [OK]    AJU: obtener() devuelve siempre la misma instancia
== P5. Arquitectura: el cliente no depende de clases concretas ==
  [OK]    ProcesadorInformes.java no importa paquetes de categorías concretas
  [OK]    ProcesadorInformes.java no instancia productos concretos con new
  [OK]    FabricaRegistro.java no importa paquetes de categorías concretas
  [OK]    FabricaRegistro.java no instancia productos concretos con new
  [OK]    ProcesadorInformes no compara categorías ni tipos con equals()

RESULTADO: 29 pruebas correctas, 0 fallos
```

Figura 9. Resultado de la suite de pruebas sobre la etapa 4b (4 categorías): 29 correctas, 0 fallos.

5.3 Experimento de escalabilidad: ¿cuánto cuesta evolucionar?

Con dos evoluciones no se puede saber cómo se comporta el diseño cuando el sistema crece. Se diseñó un experimento con **tres escenarios** y **cuatro diseños**, usando como métrica las **líneas añadidas o eliminadas en código ya existente** (el código que hay que reabrir, volver a revisar y volver a probar):

- **Escenario 1:** añadir una categoría con todos los tipos existentes (el caso MIXTA).
- **Escenario 2:** añadir un tipo a *todas* las categorías.
- **Escenario 3:** añadir un tipo a *una sola* categoría (el caso PEDIDO).

Rigor del modelo: Un modelo sin contrastar no prueba nada, así que antes de extrapolar se calibró contra mediciones reales del proyecto. El escenario 2 no figuraba en el enunciado: se implementó de verdad un tipo NOTA en las tres categorías, y las 18 líneas medidas coincidieron con la predicción.

Tabla 8. Calibración: líneas de código existente modificadas, modelo frente a medición.

Caso (medido sobre el código real)	Modelo	Medido
Abstract Factory: añadir MIXTA	2	2
Abstract Factory: añadir PEDIDO a ENTRADA	17	17
Abstract Factory: añadir NOTA a las 3 categorías (nuevo)	18	18
Registro por clave: añadir MIXTA	3	3
Registro por clave: añadir PEDIDO a ENTRADA	1	1
Sin patrón: MIXTA y PEDIDO juntos (diff real de los dos archivos)	10	10

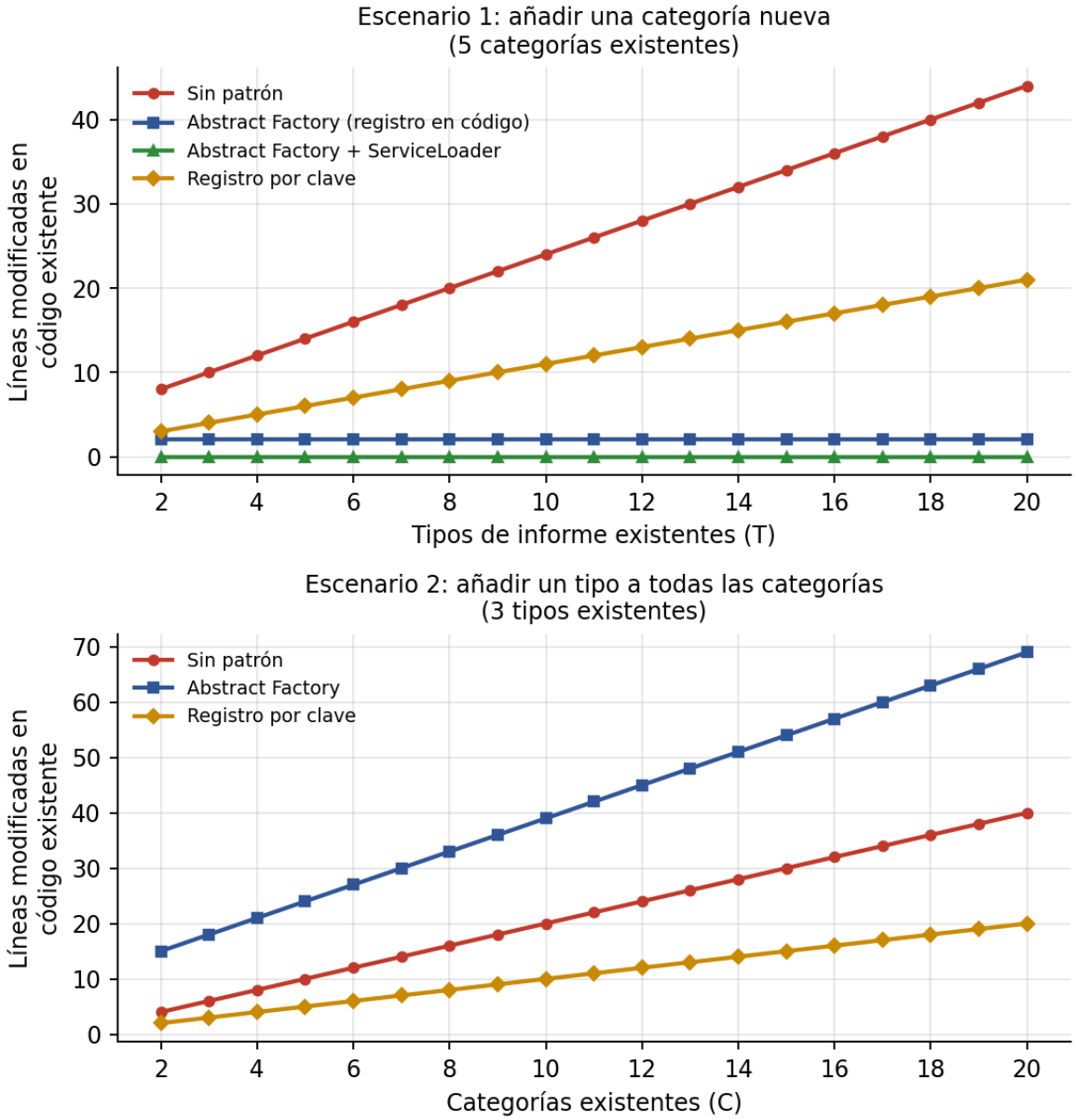


Figura 10. Crecimiento del coste de evolución según el diseño (líneas modificadas en código existente).

Tabla 9. Líneas modificadas en código existente por escenario.

Escenario	Sin patrón	Abstract Factory	AF + ServiceLoader	Registro por clave
1. Añadir categoría (C = 5, T = 3)	10	2	0	4
1. Añadir categoría (C = 5, T = 20)	44	2	0	21
2. Tipo a todas (C = 5, T = 3)	10	24	24	5
2. Tipo a todas (C = 20, T = 3)	40	69	69	20
3. Tipo a una categoría (C = 5, T = 3)	2	17	17	1

Resultados:

- **Abstract Factory gana solo el escenario 1**, y por goleada: su coste es constante (2 líneas, o 0 con *ServiceLoader*) mientras que el de los otros diseños crece con el número de tipos. Es exactamente el caso para el que fue concebido.
- **En los escenarios 2 y 3 es el diseño más costoso en líneas de código existente**, incluso más que no usar patrón (69 frente a 40 líneas con 20 categorías). El motivo es estructural: cada producto nuevo toca la interfaz abstracta y todas las fábricas (la debilidad que describe el Tema 3).
- **El registro por clave es el más barato en dos de los tres escenarios** y casi iguala al patrón en el primero cuando T es pequeño.

Límites de la métrica (importante): Contar líneas modificadas no mide todo: no recoge el *riesgo* (una línea cambiada dentro de un método monolítico con lógica de todos los informes es más peligrosa que tres líneas repartidas en clases pequeñas), ni el acoplamiento, ni las garantías del compilador. Además, los puntos de la gráfica son una extrapolación del modelo calibrado, no ejecuciones de miles de clases. Por eso el capítulo no concluye «el patrón es malo», sino que la elección depende del tipo de cambio que se espera.

5.4 Alternativa implementada: registro por clave

Para que la comparación no sea teórica, se implementó la alternativa completa (`etapa5_registro`): una tabla (categoría, tipo) → constructor, sin fábricas. Reutiliza los mismos productos de las etapas anteriores y su núcleo es este:

```
static {
    registrar("ENT", "ENTRADA", "FAC", "FACTURA", FacturaEntrada::new);
    registrar("ENT", "ENTRADA", "COM", "COMPRA", CompraEntrada::new);
    registrar("SAL", "SALIDA",  "FAC", "FACTURA", FacturaSalida::new);
    // ... una línea por combinación ...
    registrar("ENT", "ENTRADA", "PED", "PEDIDO",  PedidoEntrada::new);
}
```

Equivalencia verificada: Se ejecutaron la etapa 3 y la etapa 5 con la misma entrada y se compararon las salidas: son **idénticas byte a byte** (18 líneas, mismo hash MD5). Es decir, ambos diseños resuelven el problema igual de bien; difieren en cómo cuestan y en qué garantizan. La alternativa necesita 14 archivos y 211 líneas de producción, frente a 20 y 348 del patrón.

Lo que Abstract Factory aporta y el registro no: Una fábrica concreta está obligada por la interfaz a implementar *todos* los métodos de creación: es imposible «olvidar» un tipo en una categoría, y el compilador lo señala. En el registro, una combinación omitida solo se descubre en ejecución, como un aviso «no admite informes de tipo…». Del mismo modo, el patrón permite sustituir **una familia completa** de forma atómica (por ejemplo, una `SalidaFactory` de pruebas), algo que en el registro exige reemplazar cada entrada.

Tabla 10. Matriz de decisión entre los tres diseños.

Criterio	Sin patrón	Abstract Factory	Registro por clave
Añadir una categoría	Desfavorable (crece con T)	**Favorable** (constante)	Intermedio (crece con T)
Añadir un tipo a todas las categorías	Intermedio	Desfavorable	**Favorable**

Criterio	Sin patrón	Abstract Factory	Registro por clave
Añadir un tipo a una categoría	**Favorable**	Desfavorable	**Favorable**
Completitud de la familia verificada al compilar	No	**Sí**	No (error en ejecución)
Coherencia de la familia garantizada	No	**Sí**	No
Lógica de cada informe aislada y probable	No (un solo método)	Sí	Sí
Tamaño del código de producción	1 archivo, 46 líneas	20 archivos, 348 líneas	14 archivos, 211 líneas

5.5 Guía de decisión y recomendación

- **Abstract Factory** conviene cuando las **familias** son el eje del cambio (aparecen categorías nuevas con frecuencia) y se valora que el compilador garantice que cada familia está completa y es coherente. Es el caso típico de *toolkits* de interfaz o *drivers* de base de datos.
- **Registro por clave** conviene cuando las combinaciones son **independientes** y el eje de cambio son los tipos: añadir o quitar una combinación debe costar una línea.
- **Abstract Factory + ServiceLoader** (etapa 4) conviene cuando, además, las categorías deben poder **añadirse como módulos** sin recompilar el núcleo, y se compensa el riesgo de configuración con una suite de pruebas como la de la sección 5.2.

Recomendación para este problema. Si solo importara el coste de cambio, la mejor opción para el sistema tal como está planteado sería el registro por clave. Se mantiene *Abstract Factory* porque el enunciado lo propone y porque aporta dos garantías que el registro no ofrece (completitud y coherencia de familia); y se complementa con *ServiceLoader* y pruebas para eliminar el acoplamiento residual. Reconocer que otra solución sería más barata para un tipo de cambio concreto no debilita el análisis: es lo que distingue aplicar un patrón de entenderlo.

6. CONCLUSIONES

El desarrollo de esta actividad permitió analizar, diseñar, implementar y evaluar el patrón de creación Abstract Factory, aplicado a un sistema de procesamiento de informes en Java. A través de los diferentes ejercicios y etapas de evolución, fue posible contrastar sus fundamentos teóricos con los resultados obtenidos durante la implementación, identificando tanto sus ventajas estructurales como sus limitaciones prácticas. Asimismo, la comparación con alternativas sin patrón y con otros mecanismos de creación permitió establecer en qué escenarios su utilización representa una solución conveniente y cuáles son los compromisos que implica desde la perspectiva de la arquitectura de software.

A partir del diseño, la implementación y las pruebas realizadas, se establecen las siguientes conclusiones:

1. Aplicabilidad del patrón Abstract Factory: El patrón resuelve adecuadamente el problema planteado cuando las categorías se modelan como familias y los tipos de informe como productos relacionados. Su aplicación permite encapsular las clases concretas, mantener la coherencia de los productos creados por cada categoría y desacoplar al cliente de las implementaciones específicas. Como resultado, el componente consumidor puede operar mediante abstracciones sin depender directamente de las clases que conforman cada familia.

2. Impacto de las evoluciones sobre la arquitectura: Las etapas de evolución confirmaron la premisa fundamental estudiada en el Tema 3: incorporar nuevas familias de productos resulta favorable para Abstract Factory, mientras que incorporar nuevos productos implica un mayor esfuerzo estructural. La categoría MIXTA se incorporó mediante tres clases nuevas y una línea de registro, mientras que el tipo PEDIDO requirió modificar dos archivos existentes y adoptar una decisión adicional de diseño mediante una interfaz derivada.

3. Coste estructural y justificación de su aplicación: Abstract Factory no constituye una solución gratuita, puesto que introduce clases adicionales, niveles de abstracción e indirección en el proceso de creación. Por esta razón, su utilización no se justifica necesariamente en sistemas con pocos comportamientos

triviales. Su valor se hace más evidente cuando existen familias de productos con responsabilidades diferenciadas, comportamientos propios y una expectativa real de evolución y mantenimiento.

4. Comparación con otros patrones de creación: El análisis de las alternativas permitió establecer que los demás patrones estudiados cumplen propósitos diferentes y no sustituyen directamente la solución seleccionada. Factory Method se orienta a la creación de un producto, Builder resulta más apropiado para objetos que requieren un proceso de construcción por etapas y Singleton se utilizó como mecanismo complementario para garantizar una única instancia de las fábricas concretas. Por tanto, la selección de Abstract Factory responde a la necesidad específica de gestionar familias de productos relacionados.

5. Importancia de la medición en las decisiones arquitectónicas: El experimento desarrollado en el capítulo 5 permitió comprobar que Abstract Factory no representa necesariamente la alternativa de menor coste en todos los escenarios. Su comportamiento resulta más favorable cuando se espera incorporar nuevas familias, mientras que la adición de nuevos productos puede generar un coste superior al de una implementación sin patrón. Asimismo, la alternativa implementada mediante un registro por clave produjo una salida idéntica con menor cantidad de código y un coste inferior en dos de los tres escenarios evaluados. Esto demuestra que las decisiones de diseño deben sustentarse en las características del problema y en resultados medibles, no únicamente en los beneficios teóricos de un patrón.

6. Control y reducción del acoplamiento: El desarrollo del Ejercicio 3 permitió identificar que desacoplar el cliente no garantiza, por sí solo, eliminar todas las dependencias entre componentes, debido a que el registro central todavía conocía las categorías disponibles. La incorporación de ServiceLoader y de una prueba de arquitectura automatizada permitió avanzar hacia una solución más extensible, reduciendo esa dependencia. Adicionalmente, la suite de 29 pruebas, incluida una violación arquitectónica introducida deliberadamente, permitió comprobar que las restricciones definidas podían ser verificadas automáticamente.

7. Posibilidades de evolución funcional: Como línea de mejora, se identifica la incorporación de un mecanismo real de lectura del contenido de los archivos .txt. Esta funcionalidad podría integrarse sin modificar la estructura general de las familias, pasando el contenido leído al producto concreto correspondiente, que conservaría la responsabilidad de ejecutar su procesamiento específico. De esta manera, sería posible ampliar el comportamiento funcional del sistema manteniendo la separación de responsabilidades establecida.

8. Reutilización y verificación del diseño: La reutilización del código se abordó desde la separación de responsabilidades y se contrastó mediante pruebas automatizadas y comparaciones de resultados entre etapas. En particular, durante la etapa 4b, la categoría AJUSTE se incorporó sin modificar líneas de código Java existente. Este resultado, junto con las pruebas de comportamiento y arquitectura, demuestra la importancia de acompañar la reutilización de mecanismos de verificación que permitan comprobar que las extensiones no afectan negativamente el funcionamiento previamente establecido.

En conclusión, la actividad permitió comprobar que la calidad de una decisión arquitectónica depende de su correspondencia con el problema que se pretende resolver y del comportamiento que demuestra frente a los cambios. Abstract Factory ofrece una estructura sólida para administrar familias de productos relacionados y facilita determinadas formas de evolución, pero también introduce costes que deben evaluarse antes de adoptarlo. Los resultados obtenidos evidencian que no existe un patrón universalmente superior, sino alternativas cuyo valor depende de los requisitos, la naturaleza de las modificaciones previstas y los objetivos de mantenibilidad del sistema.

Finalmente, el ejercicio permitió integrar los conceptos de diseño orientado a objetos con prácticas de implementación, pruebas automatizadas, análisis comparativo y evaluación arquitectónica. De esta manera, se reafirma que aplicar un patrón de diseño no consiste únicamente en reproducir una estructura conocida, sino en comprender su propósito, justificar su utilización, reconocer sus limitaciones y verificar mediante evidencia técnica que aporta beneficios reales a la solución desarrollada.

7. BIBLIOGRAFÍA

Las siguientes referencias corresponden a las fuentes bibliográficas y materiales académicos consultados para fundamentar el desarrollo de la actividad, en particular los patrones de creación, la comparación entre ellos y los principios de diseño aplicados.

- Debrauwer, L. (2013). *Patrones de diseño en Java*. Ediciones ENI.
- C2. (2007). *Abstract Factory vs Factory Method*. http://wiki.c2.com/?AbstractFactoryVsFactoryMethod
- Gamma, E. (2002). *Patrones de diseño* (2.ª ed.). Pearson Educación.
- Martin, R. C. (2002). *Agile software development: Principles, patterns, and practices*. Prentice Hall.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- Universidad Internacional de La Rioja. (s. f.). *Tema 3. Patrones de creación* [Material didáctico]. En *Reutilización de Software*. UNIR.
- Universidad Internacional de La Rioja. (s. f.). *Tema 4. Patrones de creación (II)* [Material didáctico]. En *Reutilización de Software*. UNIR.
- Welicki, L. (2008). *Patrones de fabricación: Fábricas de objetos*. Microsoft. https://msdn.microsoft.com/es-es/library/bb972258.aspx

8. ANEXO: ENTREGA DEL CÓDIGO Y EJECUCIÓN

Repositorio: https://github.com/AlejoTechEngineer/abstract-factory-procesador-informes. La entrega consiste en este documento en PDF y el enlace al repositorio, donde el código puede copiarse y ejecutarse siguiendo el README.

El proyecto Java completo está en el repositorio (carpeta Actividad1_Patrones_Creacion_codigo). Cada etapa es independiente y tiene su propio main:

```
etapa1_base/     Ejercicio 2: sistema base (ENTRADA/SALIDA x FACTURA/COMPRA)
etapa2_mixta/    Ejercicio 3: + categoría MIXTA
etapa3_pedido/   Ejercicio 3: + tipo PEDIDO en ENTRADA   (versión del enunciado)
etapa4_servicios/ Cap. 5: categorías con ServiceLoader + suite de pruebas
etapa4b_ajuste/  Cap. 5: + categoría AJUSTE sin modificar Java existente
etapa5_registro/ Cap. 5: alternativa por registro (salida idéntica a etapa 3)
experimento/     Cap. 5: escala.py (modelo de costes calibrado) y gráfica
sinpatron/       Ejercicio 1: versiones sin patrón (base y evolucionada)
uml/             Diagramas UML (PNG y fuente Graphviz .dot)
compilar_ejecutar.sh   Compila y ejecuta una etapa
```

```
$ ./compilar_ejecutar.sh etapa3_pedido        # requiere JDK 8 o superior
$ ./probar.sh etapa4b_ajuste                  # suite de 29 pruebas automáticas
$ python3 experimento/escala.py               # experimento de escalabilidad
# o manualmente:
$ javac -encoding UTF-8 -d out $(find etapa3_pedido/src -name "*.java")
$ java -cp out informes.app.Main
```

9. AGRADECIMIENTO

Finalmente, deseo expresar mi sincero agradecimiento al profesor Deivis Eduard Ramírez Martínez por el acompañamiento y los conocimientos compartidos a lo largo de la asignatura Reutilización de Software.

El desarrollo de esta actividad permitió comprender los patrones de creación no solo como una teoría, sino como una herramienta cuya conveniencia debe evaluarse con evidencia: se diseñó, implementó, evolucionó y midió una solución, y se contrastó con alternativas para decidir con criterio cuándo aplicar cada una.

Muchas gracias por la orientación y por la oportunidad de profundizar en estos temas.

Respetuosamente,

Alejandro De Mendoza